

**Sigurado: A Dual-Biometric Sequential Access and Materials
Accountability System for a Laboratory Stockroom**

Gerjhon Aballa, John Lloyd Abragan, and John Adam A. Cuenca

Department of Computer Engineering

CPE 048: Embedded Systems — COC-FA-CPE4-01

[INSTRUCTOR NAME]

[DUE DATE]

Contents

CHAPTER 1: INTRODUCTION	1
1.1 Background of the Study	1
1.2 Statement of the Problem	3
1.3 Objectives of the Study	6
1.4 Significance of the Study	7
1.5 Scope and Limitations	8
1.6 Definition of Terms	9
 CHAPTER 2: SYSTEM DESIGN	 11
2.1 System Overview	11
2.2 System Architecture	12
2.2.1 Overall System Architecture	12
2.2.2 Microcontroller Architecture	14
2.2.3 Memory Architecture	14
2.2.4 Input and Output Architecture	15
2.2.5 Communication Architecture	16
2.3 Block Diagram	17
2.4 Hardware Components	17
2.5 Hardware Architecture Design	18
2.6 Data Flow	20
2.6.1 Steps in One Complete Use	21
2.6.2 The ESP32 Firmware	22
2.6.3 The Backend	24
2.6.4 The Frontend	26
2.6.5 How the Chip Runs the Instructions	28

2.7 What a Person Sees at the Reader	30
2.8 The Web Console	33
2.8.1 The Public Page	33
2.8.2 What Each Role Sees	34
2.8.3 The Record	35
2.8.4 On a Phone	36
CHAPTER 3: TESTING AND PERFORMANCE	38
3.1 Unit and Integration Testing	38
3.2 System Testing	39
3.2.1 Testing the Assembled Readers	39
3.3 Test Results	40
3.4 System Requirements	42
3.5 Performance Analysis	44
3.5.1 Use of Chip Resources	44
3.5.2 Timing	44
3.5.3 Comparison With Published Systems	45
CHAPTER 4: CONCLUSION	47
4.1 Problems Encountered	47
4.1.1 Slot Numbers Meant Different People	47
4.1.2 A Reader Cannot Be Trusted to Decide	47
4.1.3 One Unknown Record Emptied the Whole Page	47
4.1.4 Settings Were Cut Without Warning	47
4.1.5 Electrical Problems That Only Appear in Hardware	48
4.1.6 The Door Reader May Not Reach the Wi-Fi	48
4.2 Troubleshooting	48

4.2.1 Finding the Empty Page Fault	48
4.2.2 Always Start With Something That Works	49
4.2.3 Checking Code Before Flashing	49
4.3 Solutions and Conclusions	49
4.3.1 What We Fixed	49
4.3.2 Conclusion	50
4.3.3 Recommendations	51

List of Figures

1	One complete use, followed through the readers, the backend, and the frontend	13
2	The cabinet and the door kiosk, each beside the frame it is built on	17
3	The cabinet with its service panel removed, showing the electronics bay above the materials bins	19
4	Inside the reader box with the back plate removed, showing the board, sensor, relay and lock wiring	20
5	The cabinet reader's firmware: start-up, then one pass of the loop	23
6	How the backend decides a scan from the door or the cabinet	25
7	The checkout page, from opening it to the record being saved	27
8	The fetch, decode, and execute cycle, with an example of each kind of instruction from the reader firmware	29
9	The public page, which explains the system without exposing any record	33
10	The administrator dashboard, showing live sessions and the decisions as they arrive	35
11	The trail as a spreadsheet, with grants and refusals in the same list	36
12	The site on a phone: the public page, the dashboard, and the checkout form where the one-time code is entered	37

List of Tables

1	How the chip uses its memory	15
2	What is connected to which pin	16
3	Parts used and what each one does	18
4	What the door reader displays	31
5	What the door reader displays while a finger is being signed up	31
6	What the cabinet reader displays	32
7	What we tested and what happened	41
8	What the system must do	43
9	Timings we set, and timings still to be measured	45

CHAPTER 1: INTRODUCTION

1.1 Background of the Study

Schools lend laboratory equipment to their students and staff, and keeping track of what has gone out is a recognised management problem (Cepeda & Saludes, 2025). Cepeda and Saludes (2025) examined equipment lending at a Philippine university and reported that handling the process by hand left the institution weak in three areas at once: operational efficiency, data accuracy, and accountability. Those three weaknesses are the reason such institutions move to recorded, automated lending (Cepeda & Saludes, 2025). The gap survives because entry to a room and the record of what left it are governed by two separate mechanisms that never verify one another (Cepeda & Saludes, 2025).

Records maintained by hand also drift away from the physical stock they are meant to describe (Linuwih & Handayati, 2025). Linuwih and Handayati (2025) counted stock against the written record in a warehouse and found that 52.85 percent of product types did not match during a single stocktake. A written record that disagrees with the shelf in more than half of cases cannot settle a question about who holds what (Linuwih & Handayati, 2025). Physical keys carry a separate weakness of their own (Alrawili et al., 2024). Alrawili et al. (2024) observe that anything a person carries, as opposed to something a person is, can be handed to somebody else. A key is therefore able to be lent, copied, or lost, after which the lock records only that somebody held a key rather than who that person was (Alrawili et al., 2024).

Biometric identification answers that weakness directly, because a fingerprint belongs to the person rather than being carried by them (Alrawili

et al., 2024). Alrawili et al. (2024) reviewed how such systems are chosen and found that the choice is always a balance. A system has to be secure, but it also has to be convenient and robust, or it will not survive contact with the people who use it (Alrawili et al., 2024). Fingerprints sit at a workable point on that balance, being inexpensive to read and quick enough to complete a scan in roughly one second on low-cost hardware (Alsayaydeh et al., 2025).

Fingerprint locks built on low-cost microcontrollers are now a well documented application of this hardware (Rahman et al., 2026). Alsayaydeh et al. (2025) built one using an ESP32, a small computer chip with built-in Wi-Fi. Their lock read a finger in roughly one second and released the door about five seconds later (Alsayaydeh et al., 2025). It admitted the wrong person 1.32 percent of the time, which is a good result (Alsayaydeh et al., 2025). It also turned away the right person 26.32 percent of the time, so nearly one legitimate user in four had to present a finger again (Alsayaydeh et al., 2025). That rejection figure is the convenience side of the balance described by Alrawili et al. (2024), on which a system that is not convenient will not survive contact with its users.

A single reader also carries a limit that no improvement in sensor accuracy can remove, since one reader verifies only the credential presented to it and nothing about the person who may have entered alongside its user (Mohammed et al., 2025). Researchers have answered this by verifying a person more than once rather than by trying to perfect a single check (Mohammed et al., 2025). Mohammed et al. (2025) tested a system built this way and measured a 68 percent drop in fraudulent access. The gain came from requiring more than one independent confirmation

before granting entry, not from a more accurate sensor (Mohammed et al., 2025). Their framework still answered in 235 milliseconds on average, so the extra check did not make the system slow (Mohammed et al., 2025).

The record itself must also resist alteration, since a saved list is only useful if nobody can revise it after an incident (Koisser & Sadeghi, 2023). Koisser and Sadeghi (2023) studied this problem across connected devices and describe tampering as the central weakness of digital records, which is why their scheme is built to make any modification detectable rather than merely discouraged. Read together, these findings describe one design: verify a person twice and bind the second check to the first (Mohammed et al., 2025), then hold the result where it cannot be edited (Koisser & Sadeghi, 2023).

1.2 Statement of the Problem

A school stockroom cannot prove who took its materials, because the accountability its records are meant to provide is precisely what manual handling fails to deliver (Cepeda & Saludes, 2025). Two weaknesses produce this failure. A single reader verifies only the credential presented to it, so a person admitted alongside an authorized user is never evaluated and never recorded, which is the weakness Mohammed et al. (2025) address by requiring more than one confirmation. Written records then disagree with the physical stock they describe in more than half of the product types examined (Linuwih & Handayati, 2025), and moving those records onto a computer does not repair the problem, since a list that can still be revised afterwards carries the same weakness as a paper one (Koisser & Sadeghi, 2023). The institution is therefore left unable to show either that its list of names is complete or that nothing on it was altered

later, which are the two properties an accountable record requires (Koisser & Sadeghi, 2023; Linuwih & Handayati, 2025).

Existing work addresses the parts of this problem separately, and the gap lies between them (Alsayaydeh et al., 2025; Cepeda & Saludes, 2025; Koisser & Sadeghi, 2023; Mohammed et al., 2025). Fingerprint locks on low-cost microcontrollers guard a single control point (Alsayaydeh et al., 2025). Requiring more than one confirmation is known to cut fraudulent access, but that result was obtained by combining modalities at one point on server-class hardware rather than by sequencing them across two physical locations (Mohammed et al., 2025). Tamper-evident logging is well developed, and recent work brings it within reach of resource-limited devices (Yagiz et al., 2026), yet it is treated as a logging problem rather than one bound to a physical access decision (Koisser & Sadeghi, 2023). Per-device credentials and replay resistance have likewise been demonstrated on ESP32-class hardware in isolation (Lkhluf et al., 2026). Equipment borrowing has been moved onto computers, but the written claim is not tied to a physical opening (Cepeda & Saludes, 2025), which is why records still drift from the shelf (Linuwih & Handayati, 2025). Across the work reviewed here (Alsayaydeh et al., 2025; Cepeda & Saludes, 2025; Koisser & Sadeghi, 2023; Linuwih & Handayati, 2025; Lkhluf et al., 2026; Mohammed et al., 2025; Yagiz et al., 2026), no single reported system joins these into one scheme: sequential, identity-bound verification across two control points, running on low-cost microcontrollers, with an unalterable record that binds what was taken to the opening that allowed it.

Specifically, this study seeks to answer the following questions:

1. Can sequential, identity-bound verification across two control points be

implemented on low-cost microcontrollers, given that the reduction in fraudulent access reported by Mohammed et al. (2025) was obtained on server-class hardware?

2. What session lifetime binds the second check to the first without obstructing legitimate use, given the rejection rate reported for single-reader fingerprint locks (Alsayaydeh et al., 2025) and the idle-timeout range recommended for high-value applications (OWASP Foundation, 2026)?
3. Can an access record be held unalterable by every role, including administrators, on a system of this scale, where tamper-evident logging has so far been treated separately from the access decision itself (Koisser & Sadeghi, 2023; Yagiz et al., 2026)?
4. Can a written record of borrowed items be bound to a physical opening, closing the gap left when borrowing is digitised without such a link (Cepeda & Saludes, 2025; Linuwih & Handayati, 2025)?
5. Do the processor, memory, input/output, and communication resources of the ESP32 support this combination, given the limited on-chip memory identified as the platform's standing constraint (Rahman et al., 2026)?
6. What residual weaknesses remain in a design that delegates matching to the sensor, given that low-cost fingerprint modules have been shown to be spoofable (Rad et al., 2024)?

1.3 Objectives of the Study

The general objective of this study is to design and build an embedded system that can prove who took materials from a school stockroom, and can store that proof in a form that nobody is able to change afterwards.

The specific objectives below answer the six questions above, in the same order, each stating what the study builds or measures to settle its matching question.

1. To implement sequential, identity-bound verification on two low-cost microcontroller readers, so that the cabinet opens only for the person the door admitted, and to demonstrate it on assembled hardware rather than on server-class equipment (Mohammed et al., 2025).
2. To set and enforce a door session lifetime of 120 seconds, taken from the 2 to 5 minute idle timeout recommended for high-value applications (OWASP Foundation, 2026), and to close it automatically.
3. To hold every scan, whether allowed or refused, in a record that offers no route to edit or delete it for any role, administrators included (Koisser & Sadeghi, 2023).
4. To bind a written list of borrowed items to a physical opening by requiring a one-time code that only the opening cabinet displays (Cepeda & Saludes, 2025).
5. To document how the processor, memory, input/output, and communication resources of the ESP32 support this combination, and to measure what the finished program costs against the limited

on-chip memory that Rahman et al. (2026) identify as the platform's standing constraint.

6. To state the residual weaknesses of the design, including those inherited by delegating fingerprint matching to a sensor module of a class shown to be spoofable (Rad et al., 2024).

1.4 Significance of the Study

This study is useful to the groups described below.

Students. Students who borrow equipment gain a fair and complete record. Because every scan is saved with a name and a time, a student cannot easily be blamed for something that somebody else removed. The record protects the borrower just as much as it protects the school, and it removes the argument that so often follows a missing item. Students also gain a faster way of borrowing, since there is no notebook to find and no queue behind the person writing in it.

Teachers and laboratory staff. Staff no longer need to read other people's handwriting or chase students for entries that were never made. The list builds itself as people use the room, and it can be searched in seconds rather than flipped through page by page. Staff can also see at a glance whether a reader has stopped working, instead of discovering it only when something goes wrong.

Stockroom custodians and administrators. Administrators gain something a notebook can never provide, which is proof. When an item goes missing they can see exactly who opened the cabinet, at what time, and what was logged against that opening. This speaks directly to the record inaccuracy measured by Linuwih and Handayati (2025), where more

than half of the product types checked disagreed with their written record. Administrators can also see refused attempts, which reveals whether people are trying to enter when they should not be. That second view turns a vague worry about losses into a number that can be watched over time.

The school. Laboratory equipment is expensive and school budgets are limited. Better records mean fewer unexplained losses, and they support better decisions about what needs replacing and how much to buy. Over several terms, the record also shows which items are used heavily and which sit untouched.

Future researchers. This study documents what worked and what did not, including a fault discovered during testing and the measurements still to be taken. Other students can reuse the design, extend it to more rooms, or take the honest account of its limits as a starting point and avoid repeating the same mistakes.

1.5 Scope and Limitations

This study covers one door and one storage cabinet inside a single school stockroom, served by two reader units and one server. A server is the computer that stores all the records and makes the decisions about who may enter. The system handles signing up a fingerprint, the two scans, the borrowing record, the saved history, and text message alerts. It also shows different information to students, teachers, and administrators, so that a student sees only their own records while an administrator sees everything.

The study carries the following limits, and every claim made later in this paper should be read against them.

1. **Performance was not measured in numbers.** Both readers

were built, wired, flashed and exercised through the complete door-to-cabinet sequence, so the system is demonstrated to work. What the study does not provide is quantitative performance data: scan time, unlock time and error rates were not recorded over a controlled set of trials, so no such figure is claimed here.

2. **The sensor does the matching.** Fingerprints are stored and compared inside the sensor itself, never on our chip. The system is therefore only as accurate as the sensor, and it has no way of telling a real finger from a fake one. Rad et al. (2024) showed that copies made from seaweed jelly can fool this kind of sensor.
3. **People must sign up twice.** Once at the door and once at the cabinet, because each sensor keeps its own separate list of fingers.
4. **The borrowed list is still typed by a person.** The system can prove that the cabinet opened and who opened it. It cannot check whether the list of items typed afterwards is complete or honest.
5. **Text messages were not sent over a real network.** They were tested using a stand-in program on the computer rather than a real SIM card and a real phone line.
6. **One room only.** The design was not tested across several rooms or with several cabinets running at the same time.

1.6 Definition of Terms

The terms below are listed in alphabetical order and are defined as they are used in this study.

Audit trail The saved list of every scan and every decision the system has made.

Embedded system A small computer built into a device to do one particular job.

ESP32 The chip used in this project. It is a small computer with built-in Wi-Fi.

Fingerprint sensor The part that reads a finger and compares it against a list stored inside itself.

Memory Space inside the chip that holds the program and the data it is working on.

Relay An electric switch that lets a small chip control a much stronger power supply.

Server The main computer that keeps every record and decides who may enter.

Session A short time window, 120 seconds in this study, that opens when somebody scans at the door.

Solenoid lock An electric lock that opens while power is sent to it and locks again when the power stops.

Tailgating Walking into a room behind somebody who has just opened the door, without scanning a finger.

CHAPTER 2: SYSTEM DESIGN

2.1 System Overview

The system has three parts: a reader at the door, a reader at the cabinet, and one server. Each reader is an ESP32 chip with a fingerprint sensor, a small screen, and an electric lock.

The readers never decide anything by themselves. When someone scans a finger, the sensor finds a match and gives back a slot number. The reader sends that number to the server. The server decides, saves the result, and sends back a yes or no.

This matters because a reader hangs on a wall where anyone can reach it. If a reader could decide on its own, someone could open it and fool it. The server sits in a locked place, so the decision stays safe.

Access happens in two steps. A finger at the door unlocks the door and starts a 120-second session for that one person. A finger at the cabinet opens it only if it belongs to the same person, and only while the session is still running. A registered finger with no session is refused, and the refusal is saved. This is the double check that Mohammed et al. (2025) found cuts fake entries.

The 120-second figure is not arbitrary. OWASP Foundation (2026) recommend an idle timeout of 2 to 5 minutes for high-value applications, against 15 to 30 minutes for low-risk ones, and a stockroom of shared equipment belongs in the first group. We took the strictest end of that range. Two minutes is long enough to cross a room and reach the cabinet, and short enough that a person who walks away leaves nothing useful behind them.

When the cabinet opens, it shows a one-time code on its screen.

The website will not accept a borrow record without that code. So every written record matches a real opening. Reader records can never be edited or deleted. This is the protection Koisser and Sadeghi (2023) say a log needs.

2.2 System Architecture

The architecture is described at five levels. The first covers how the separate parts connect to one another. The four that follow look inside a single reader, at its processor, its memory, the pins it uses to reach the outside world, and the network it speaks over.

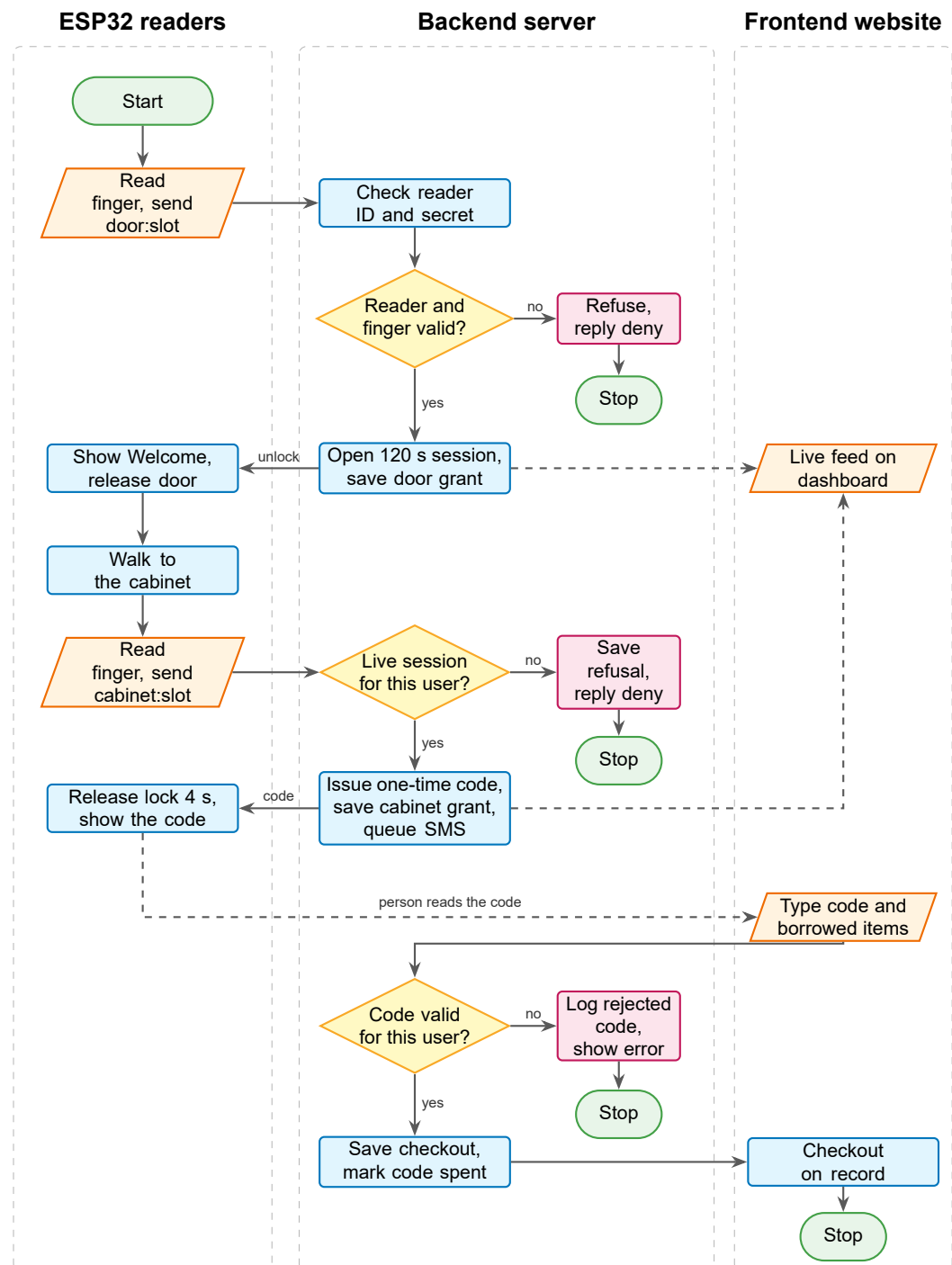
2.2.1 Overall System Architecture

The system is three programs working together. The ESP32 firmware runs on the two readers and handles the sensor, the screen, and the lock. The backend is a server written in Rust that holds the database and makes every decision. The frontend is a website, built with SvelteKit, where people sign in, enter what they borrowed, and read the record. The readers talk only to the backend, never to each other and never to the website.

Figure 1 follows one complete use through all three, with one lane for each program. Solid arrows are requests and replies. Dashed arrows are things that happen without anybody asking: the live feed updating on the dashboard, and the person carrying the code from the cabinet screen to the website. Every decision diamond sits in the backend lane, because the readers only report what they read and act on the answer.

Figure 1

One complete use, followed through the readers, the backend, and the frontend



If the door reader is out of Wi-Fi range, the cabinet reader can share its own connection with it. This bridge is built into the cabinet firmware but is switched off by default, since it is only needed where the door is too far from the router.

2.2.2 Microcontroller Architecture

Both readers use the ESP32. The chip carries two processors, each able to run at up to 240 MHz, and is rated at 1079.96 CoreMark in that configuration (Espressif Systems, 2025). Rahman et al. (2026) reviewed the platform and identified the combination of dual-core processing, integrated radios and peripheral support as its principal strength, while naming limited on-chip memory and real-time constraints as its standing weaknesses.

The second processor is what makes the workload here practical. The fingerprint sensor is slow relative to the processor, so one core can wait on the sensor while the other holds the network connection open.

2.2.3 Memory Architecture

The chip provides 448 KB of ROM for start-up code, 520 KB of on-chip RAM for running programs, and two smaller low-power regions of 8 KB each, with the application image held in external flash and executed through a cache (Espressif Systems, 2025). Limited on-chip memory is the constraint Rahman et al. (2026) single out as the platform's most persistent difficulty.

Table 1 shows how this project uses those regions. Our program fills about 81 percent of the 1.2 MB partition set aside for it, which leaves room to grow but not a great deal of it.

One design choice removes the memory pressure that Rahman et al.

(2026) describe. Fingerprints are never stored on our chip; they stay inside the sensor, which returns only a slot number. Fingerprint data is the largest structure the reader would otherwise hold, so leaving it on the sensor keeps the 520 KB of RAM (Espressif Systems, 2025) comfortable rather than tight.

Table 1

How the chip uses its memory

Storage	Size	What we use it for
ROM	448 KB	Start-up code. We do not change it.
RAM	520 KB	The running program and its working data.
Flash	1.2 MB	Our program. About 81% full.
Sensor	Inside sensor	Fingerprints. Our chip never reads them.

2.2.4 Input and Output Architecture

The chip exposes 34 programmable pins, three serial ports capable of up to 5 MBaud, two I2C interfaces and four SPI controllers (Espressif Systems, 2025). Sigurado uses three of those classes and leaves SPI unused.

Table 2 lists what we connected where. The fingerprint sensor is given its own serial port because it exchanges packet messages continuously. The screen uses I2C, which carries a display on two wires (Espressif Systems, 2025). The lock, button, and light need only single pins.

Table 2*What is connected to which pin*

Part	Connection	Door	Cabinet
Fingerprint sensor	Serial port 2	16, 17	16, 17
Screen	I2C	21, 22	21, 22
Sign-up button	Pin (in)	4	4
Status light	Pin (out)	2	2
Lock switch	Pin (out)	5	5
Door closed sensor	Pin (in)	—	18
Buzzer	Pin (out)	—	19
Text message unit	Serial port 1	—	26, 27, 25

The lock pin is the only one that moves something physical. The lock runs on its own 12 V power, not from the chip. A relay acts as the switch between them. The door lock stays open for 3 seconds and the cabinet lock for 4. The cabinet also has a small switch that tells us whether the door really closed.

2.2.5 Communication Architecture

Each reader joins the school Wi-Fi and sends messages to the server. Each reader has its own ID and password. It does not use a person's login.

This matters. If someone steals a reader and opens it, they only get that reader's password. They cannot pretend to be a person, and we can change that one password without touching the other reader. Rahman et al. (2026) name security setup as a common weak spot on this chip, and this is what it looks like in practice.

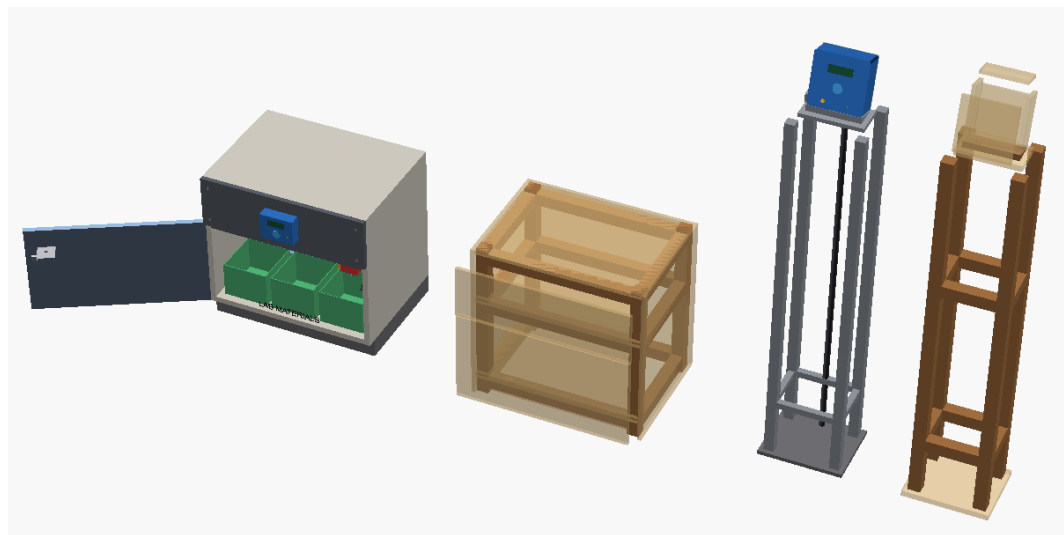
Each reader also says “I am still here” every 30 seconds. That is how the website shows which readers are working. All clocks are set from the internet, so every record has a matching time.

2.3 Block Diagram

Figure 2 shows the two physical units the system is built from. On the left is the materials cabinet, with its reader set into the panel above the bins. On the right is the door kiosk, a stand that holds the door reader at hand height beside the entrance. Each is drawn next to the lumber frame it is built on, so the finished unit and its skeleton can be compared directly.

Figure 2

The cabinet and the door kiosk, each beside the frame it is built on



2.4 Hardware Components

Table 3 lists every part used and the job it does. Quantities are per reader unless the entry says otherwise. Nothing on the list is rare or difficult to source, which was deliberate. A part that cannot be replaced from a local shop is a part that ends the project on the day it fails.

Table 3*Parts used and what each one does*

Part	What it does
ESP32 board	The small computer. Runs the program and connects to Wi-Fi. One per reader.
Fingerprint sensor (R307 or AS608)	Reads a finger and checks it against its own stored list. Fingerprints never leave it.
Small screen (16 by 2)	Shows messages. On the cabinet it also shows the one-time code.
Relay and 12 V lock	The relay is a switch. The lock runs on its own power, not the chip's.
Door switch	Cabinet only. Tells us if the door really closed.
SIM800L text unit	Cabinet only. Sends a text when the cabinet opens or someone is refused.
Buzzer, light, button	The button starts sign-up. The buzzer warns if the door is left open.

2.5 Hardware Architecture Design

The cabinet and the reader box were drawn as 3D models before building. The cabinet has three shelves behind locked doors. All the electronics sit in a hidden space behind a screwed panel, so users never see them.

Figure 3

The cabinet with its service panel removed, showing the electronics bay above the materials bins

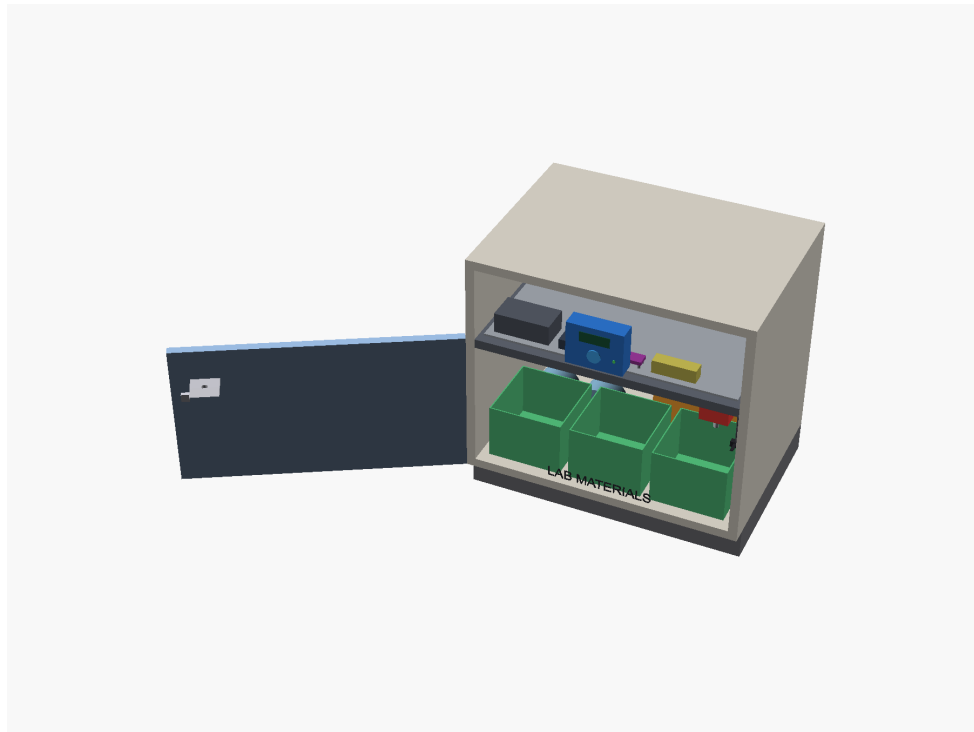
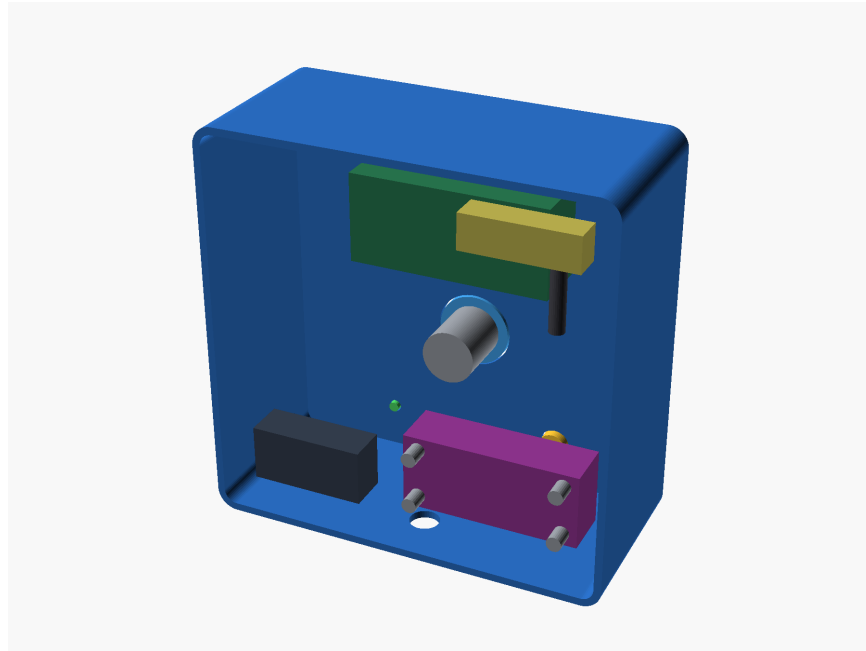


Figure 4 looks inside the reader box itself, with the back plate taken off. This is the view used to check that every part fits and to plan the wiring before anything was cut.

Figure 4

Inside the reader box with the back plate removed, showing the board, sensor, relay and lock wiring



Wiring follows Table 2. Three rules guide the layout. First, the lock gets its own power supply, because it pulls a lot of current when it opens and would otherwise restart the chip. Second, the text unit also gets its own supply, for the same reason. Third, one wire on the text unit needs a voltage reducer, because the chip's signal is too strong for it.

2.6 Data Flow

Data passes through three programs on its way from a finger to the record: the firmware on the ESP32 readers, the backend server, and the frontend website. This section first lists the steps of one complete use, then follows the data through each program in turn, with a flowchart for each. It ends one level lower, with the instructions the chip itself executes.

2.6.1 Steps in One Complete Use

The sequence below traces one successful use from beginning to end, from the first finger at the door to the borrowed items appearing in the record. It is the same path drawn in Figure 1.

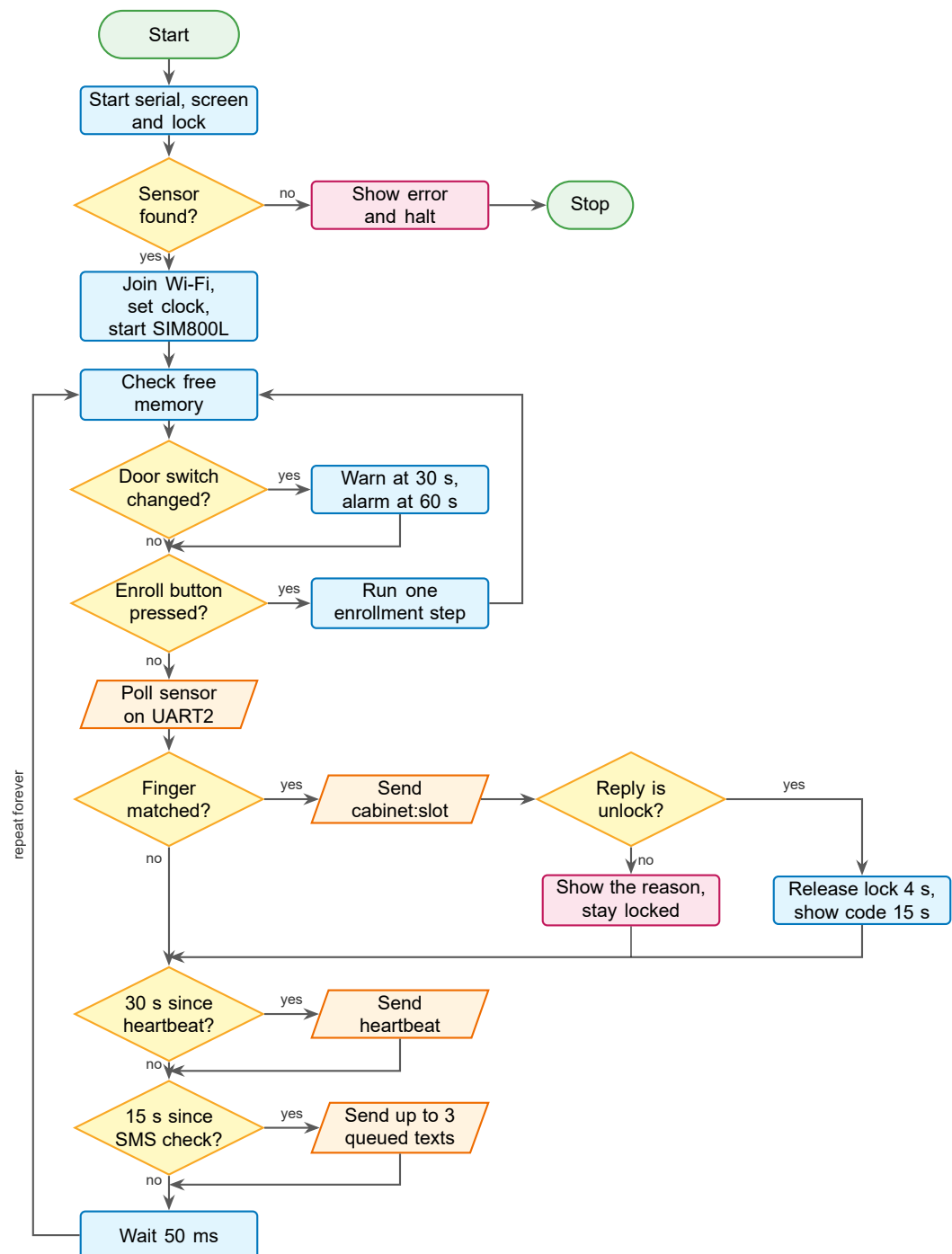
1. Someone puts a finger on the door sensor. The sensor finds a match and gives back a slot number.
2. The door reader sends that number, plus its own ID and password, to the server.
3. The server looks up the person, starts a 120-second session, saves the record, and replies “allowed”.
4. The door reader opens the lock for 3 seconds and shows a message.
5. Someone puts a finger on the cabinet sensor. The cabinet reader sends its own request.
6. The server checks whether this is the same person who has the running session. If yes, it makes a one-time code and saves the record. If no, it refuses and saves that too.
7. The cabinet releases its lock for 4 seconds and shows the code. The server queues a text message, which the cabinet sends on its next check. If the door is left open, the buzzer sounds.
8. The person types the code and the borrowed items into the website. The server accepts it only if the code is still valid.

2.6.2 The ESP32 Firmware

Each reader runs one program that starts once and then repeats a single loop for as long as it has power. Figure 5 shows the cabinet reader, which has the most to do. The door reader runs the same loop without the door switch and the text messages. The only way out of the program is at start-up: a reader whose fingerprint sensor does not answer shows an error and stops there, since it has no work to do without one.

Figure 5

The cabinet reader's firmware: start-up, then one pass of the loop



Each pass of the loop asks a fixed list of questions in the same order: has the door switch changed, was the enroll button pressed, is there a finger on the sensor, is a heartbeat due, and are text messages due. Most passes answer no to all of them and end with a 50 ms wait, so the loop runs about twenty times a second. The timers are kept by comparing the chip's millisecond clock against the last time each job ran, rather than by stopping to wait for them.

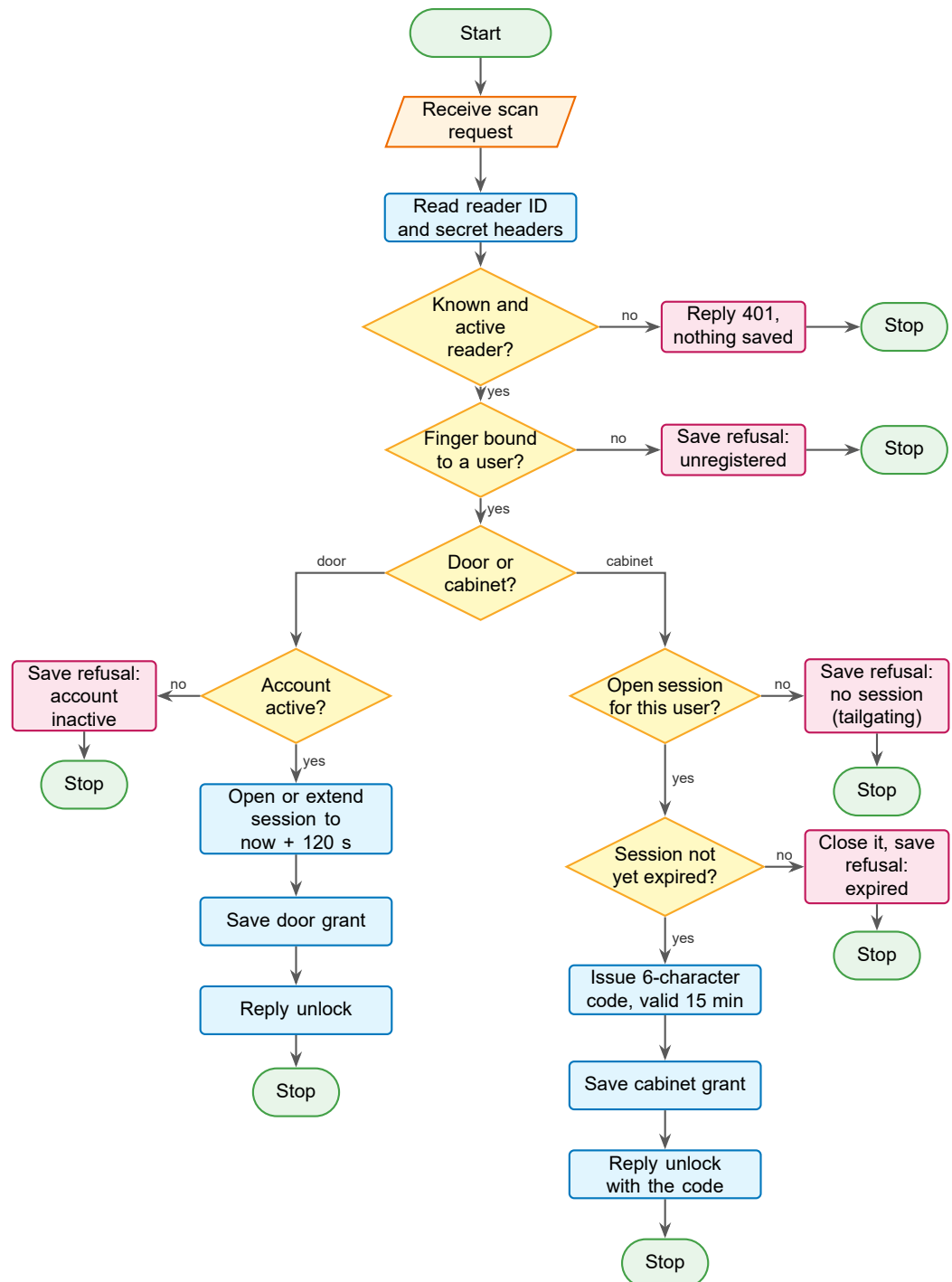
Two steps do stop the loop while they run: holding the lock open for 4 seconds, and holding a message on the screen. During those moments the reader cannot take another finger. The Wi-Fi connection is not affected, because the Wi-Fi stack runs on the chip's other core.

2.6.3 *The Backend*

Every scan from either reader arrives at the same address on the server, and Figure 6 shows what the server does with it. The first two checks apply to both readers: the request must come from a known reader with the right secret, and the finger must belong to a registered person. After that the path splits by which reader sent the scan.

Figure 6

How the backend decides a scan from the door or the cabinet



A door scan only has to confirm that the account is active, then opens a session that ends 120 seconds later. A cabinet scan has to find an open session belonging to the same person. There is no separate “wrong person” check, because the server looks sessions up by the person who just scanned. If someone else opened the door, the person at the cabinet simply has no session, and the refusal is recorded as possible tailgating.

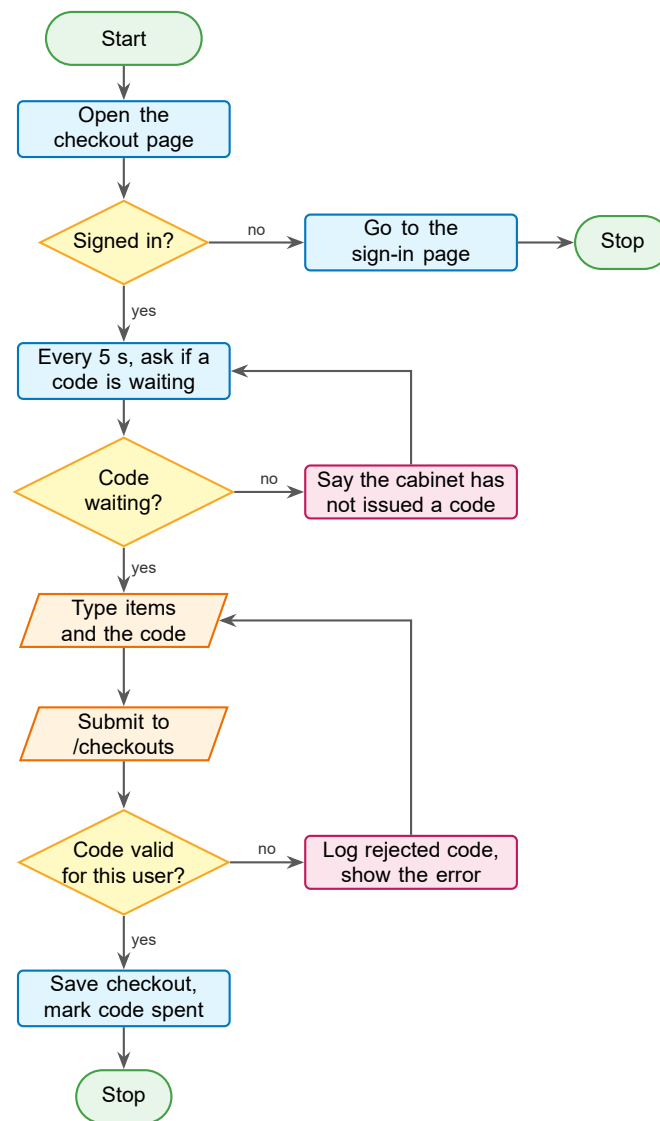
Every box in the figure that says “save” writes one row to the record. The server has no route that edits or deletes those rows. Each new row is also pushed straight to any dashboard that is open, and, if it is an opening or a refusal, a text message is queued for the people subscribed to that alert.

2.6.4 The Frontend

The website never talks to a reader. Everything it shows comes from the backend, and everything a person types goes back to it. Figure 7 shows the checkout page, which is where the cabinet’s one-time code is used.

Figure 7

The checkout page, from opening it to the record being saved



While the page is open it asks the server every 5 seconds whether a code is waiting. The server answers yes or no and when the code expires, but it never sends the code itself. The only place the code can be read is the cabinet screen, so the only way to fill in the form is to have been standing at the cabinet when it opened. The server checks the code again when the

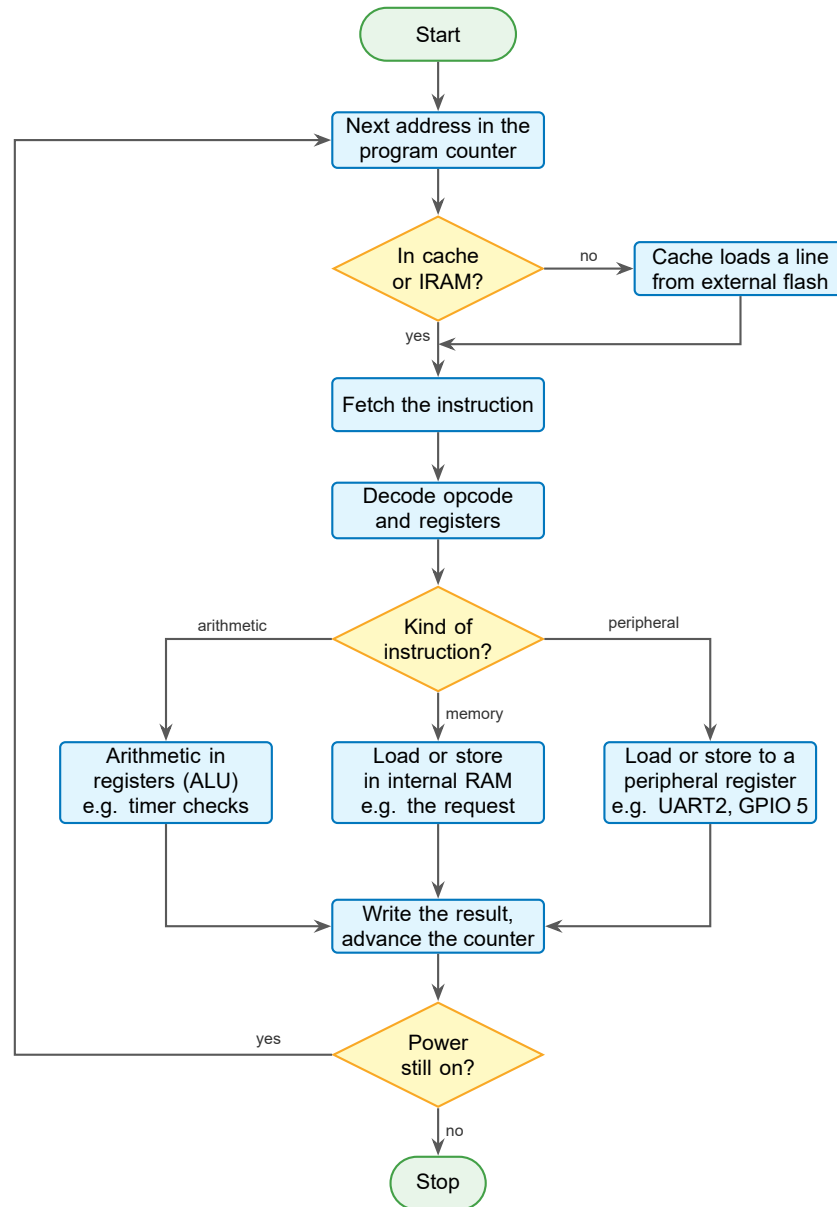
form is sent, so a code that is guessed, reused, or more than 15 minutes old is refused and the attempt is recorded.

2.6.5 How the Chip Runs the Instructions

Underneath all of this, the chip repeats one cycle. It fetches an instruction, works out what it means, and carries it out (Espressif Systems, 2025). Figure 8 shows that cycle. The program itself is stored in external flash, so an instruction that is not already in the cache has to be loaded from there first (Espressif Systems, 2025).

Figure 8

The fetch, decode, and execute cycle, with an example of each kind of instruction from the reader firmware



Three moments from the firmware show the three kinds of instruction at work.

Waiting for the sensor. Each pass of the loop reads the status of

serial port 2 to see whether the sensor has sent anything. That read is a load from a peripheral register. Most of the time nothing has arrived, and the loop moves on.

Building the message. To send a scan, the chip copies text into internal RAM piece by piece until the request is complete. These are loads and stores to memory, and this is the step that uses the most of it.

Opening the lock. Releasing the lock is one store to the register that drives pin 5. The chip then waits 4 seconds and makes one more store to close it again.

2.7 What a Person Sees at the Reader

The person standing at the door has one screen and two lines of sixteen characters to work with. That limit shapes every message: there is no room to explain, so each line has to say one thing. The wording below is taken from the firmware exactly as it is displayed.

Table 4 lists what the door reader shows. When nothing is happening it rests on its idle message, so anybody walking up already knows what to do without being told.

Table 4*What the door reader displays*

Situation	Top line	Bottom line
Waiting, nobody inside	Sigurado door	Scan finger
Waiting, session live	Sigurado door	session 97s
Asking the server	Checking...	one moment
Allowed in	Welcome	their name
Finger not signed up	Not registered	Press enroll
Sensor could not read	Did not read	please try again
No network	No WiFi yet	check settings

The second row is worth a note. While a session is running, the idle screen counts it down in place of the usual prompt, so a person crossing the room can see how long they have left before the cabinet stops recognising them. The countdown is a hint rather than the rule: the server holds the real clock, and the reader only displays what it was last told.

Signing up a finger is a separate sequence, shown in Table 5. The sensor needs the same finger presented more than once to build a usable template, so the screen walks the person through it one step at a time.

Table 5*What the door reader displays while a finger is being signed up*

Step	Top line	Bottom line
First impression	Place finger	hold still
Lift off	Remove finger	hold still
Second impression	Place again	hold still
Saving it	Saving...	one moment
Code to claim it	the code	Type it on site
Claimed	All set	their name
Sensor is full	Sensor is full	ask an admin

The code in the fifth row is the reason a template cannot be bound to the wrong account. The finger belongs to nobody until somebody signs in on the website and enters that code, so an administrator cannot quietly attach a finger to a person who never presented it.

The cabinet reader carries the harder messages, because it is the one that says no. Table 6 shows those. The two refusals are worded differently on purpose. `Scan door first` means the person never came through the door, which is the tailgating case. `Scan door again` means they did, but took longer than the session allowed. Telling those apart on the screen saves the person guessing what went wrong.

Table 6

What the cabinet reader displays

Situation	Top line	Bottom line
Allowed in	Unlocked	their name
The one-time code	Your code	the code
Where to type it	Type it on the	checkout page
Cabinet open	Take what you	need, then close
No door scan	Still locked	Scan door first
Session ran out	Still locked	Scan door again
Door left open	Door is open	Please close it
Door shut again	Door closed	Thank you
Text alert going out	Sending alert	please wait

A refused person is never left without an instruction. Every refusal names the thing to do next, because a screen that only says no teaches people to rattle the handle instead.

2.8 The Web Console

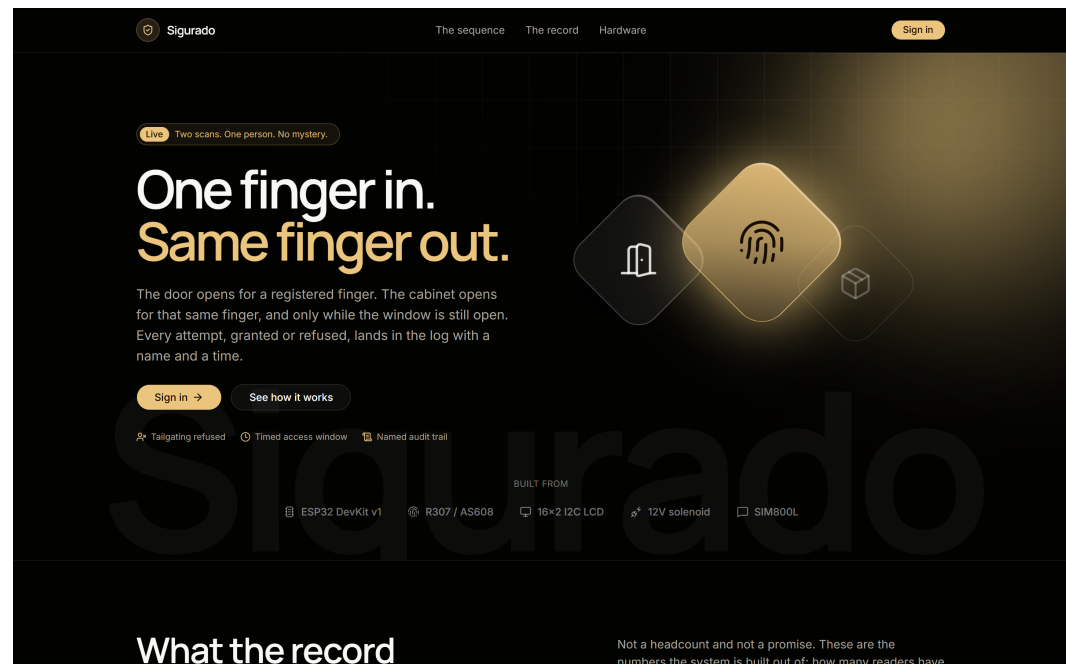
The readers handle the locks. Everything a person reads, types, or checks happens on the website. It runs in an ordinary browser and talks to the same server the readers do, so the console never sees a reader directly and cannot open anything itself.

2.8.1 The Public Page

Figure 9 shows what a visitor sees before signing in. It explains the two-scan rule and nothing else. There is no public list of people, no record of any opening, and no way in except the sign-in link. Accounts are created by an administrator, so there is no sign-up form to attack.

Figure 9

The public page, which explains the system without exposing any record



2.8.2 What Each Role Sees

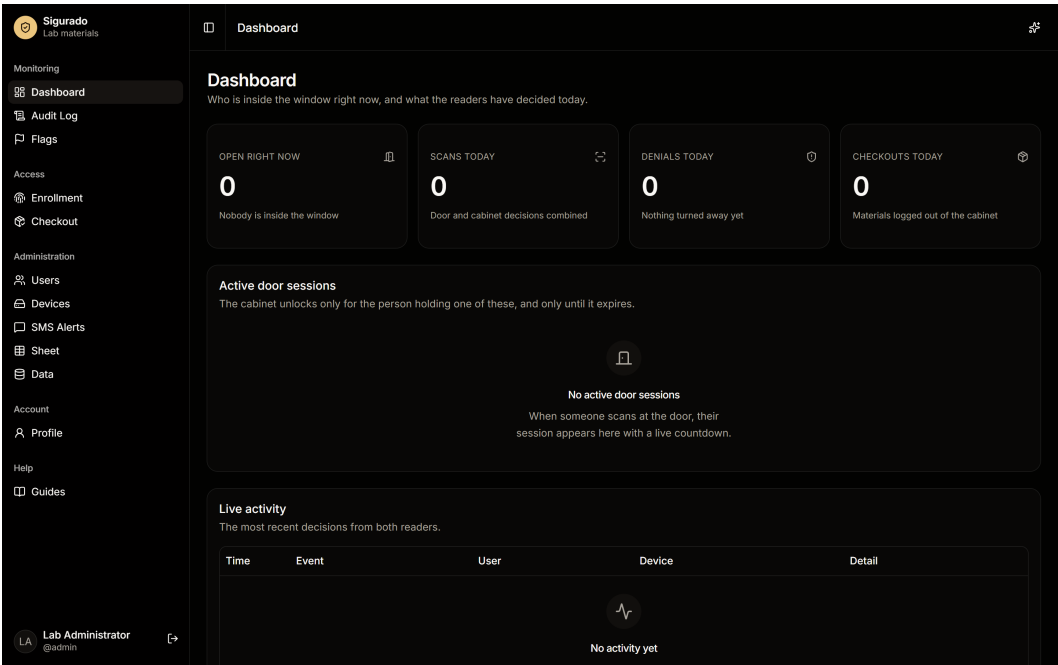
Signing in decides what the rest of the site shows. A student sees only their own scans and their own borrowed items. A member of faculty sees everything but can change nothing. An administrator runs the room: accounts, readers, alerts, and the stored data. The rule the site enforces is that a student must never see another student's record, and that nobody at all can edit a reader event.

Figure 10 shows the administrator view. The four counters along the top answer the question a custodian actually asks on walking in: is anyone inside the window right now, how many scans have happened today, how many were refused, and how much was taken. Below them sit the live door sessions, each with its own countdown, and the stream of decisions as they arrive.

The figure was captured on a quiet morning, so the counters read zero and both panels are showing their empty state. That is deliberate rather than unfinished. An empty panel that says only "no data" teaches a user nothing, so each one instead explains what will appear there and when a session or a decision will cause it. A custodian who has never seen the system before can therefore read the screen on a quiet day and still understand what it will do on a busy one.

Figure 10

The administrator dashboard, showing live sessions and the decisions as they arrive



2.8.3 The Record

Figure 11 shows the trail as a spreadsheet. Every row is something a reader reported: a door opened, a cabinet unlocked, a finger refused, an enrollment code issued. Each carries the time, the person where one is known, the finger slot, and the reader that saw it. Refusals sit in the same list as grants, which is what makes a tailgating attempt countable rather than invisible.

Figure 11

The trail as a spreadsheet, with grants and refusals in the same list

The screenshot shows the 'Sigurado' application interface. On the left is a dark sidebar with navigation links: Monitoring (Dashboard, Audit Log, Flags), Access (Enrollment, Checkout), Administration (Users, Devices, SMS Alerts, Sheet, Data), Account (Profile), and Help (Guides). The main area is titled 'Sheet' and contains a table of events. Above the table is a search bar and a '50 rows' indicator. The table has columns: When, Event, Decision, Person, Finger, Reader, and Detail. The data rows show various events like 'Door denied: finger not registered', 'Cabinet denied: finger not registered', 'Enrollment code issued', 'Fingerprint bound', 'Door opened', and 'Cabinet unlocked' with corresponding decisions (Denied, Info, Granted) and user details (Juan Ramos Dela Cruz).

	A	B	C	D	E	F	
1	When	Event	Decision	Person	Finger	Reader	Detail
2	2026-07-30 07:18:26	Door denied: finger not registered	Denied		door:41	front-door	unregistered f
3	2026-07-30 07:18:26	Cabinet denied: finger not register...	Denied		cabinet:9	cabinet-1	unregistered f
4	2026-07-30 07:18:27	Enrollment code issued	Info		door:7	front-door	enrollment co
5	2026-07-30 07:18:27	Fingerprint bound	Info	Juan Ramos Dela Cruz	door:7	front-door	fingerprint bo
6	2026-07-30 07:18:28	Enrollment code issued	Info		cabinet:3	cabinet-1	enrollment co
7	2026-07-30 07:18:28	Fingerprint bound	Info	Juan Ramos Dela Cruz	cabinet:3	cabinet-1	fingerprint bo
8	2026-07-30 07:18:29	Door opened	Granted	Juan Ramos Dela Cruz	door:7	front-door	
9	2026-07-30 07:18:29	Cabinet unlocked	Granted	Juan Ramos Dela Cruz	cabinet:3	cabinet-1	
10	2026-07-30 07:18:30	Cabinet unlocked	Granted	Juan Ramos Dela Cruz	cabinet:3	cabinet-1	
11	2026-07-30 08:29:28	Door opened	Granted	Juan Ramos Dela Cruz	door:7	front-door	
12	2026-07-30 08:29:29	Cabinet unlocked	Granted	Juan Ramos Dela Cruz	cabinet:3	cabinet-1	
13	2026-07-30 08:29:29	Cabinet denied: finger not register...	Denied		cabinet:9	cabinet-1	unregistered f
14	2026-07-30 08:29:58	Door opened	Granted	Juan Ramos Dela Cruz	door:7	front-door	
15	2026-07-30 08:29:58	Cabinet unlocked	Granted	Juan Ramos Dela Cruz	cabinet:3	cabinet-1	
16	2026-07-30 08:29:58	Cabinet denied: finger not register...	Denied		cabinet:9	cabinet-1	unregistered f
17	2026-07-30 09:01:05	Door opened	Granted	Juan Ramos Dela Cruz	door:7	front-door	
18	2026-07-30 09:01:06	Cabinet unlocked	Granted	Juan Ramos Dela Cruz	cabinet:3	cabinet-1	
19	2026-07-30 09:01:06	Door opened	Granted	Juan Ramos Dela Cruz	door:7	front-door	

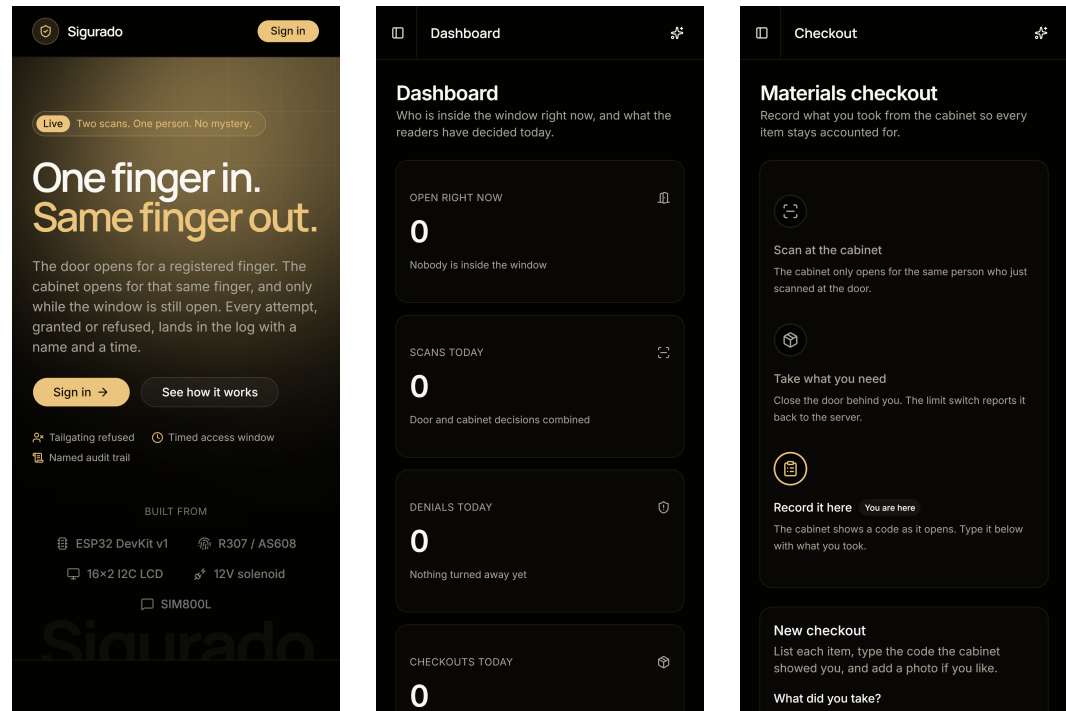
The same page exports to CSV, so the record can be kept outside the system as well. The console also runs eleven checks over the trail and lists whatever does not add up: an opening with no borrow record behind it, a finger tried repeatedly and refused, an opening outside laboratory hours, a reader that has gone quiet. These are the questions a person would have to ask by hand if the record were on paper.

2.8.4 On a Phone

Most people will reach the site on a phone, standing at the cabinet with the one-time code still on the screen in front of them. The whole console is built to work at that size, not just to survive it. Figure 12 shows three of those screens.

Figure 12

The site on a phone: the public page, the dashboard, and the checkout form where the one-time code is entered



Three things change on a small screen. The side menu folds away behind a button, so the content gets the full width. The counters stack instead of sitting in a row. The record tables scroll sideways rather than shrinking their text, because a column of times squeezed to four characters is worse than one the reader has to push across.

CHAPTER 3: TESTING AND PERFORMANCE

Testing ran at three levels. Unit tests checked the decision logic on its own. Integration tests checked that the website and the server agree with each other. System tests exercised the assembled hardware end to end, from a finger at the door to a borrowed item appearing in the record.

One limit applies to everything below. The tests establish that each behaviour happens, not how fast or how reliably it happens over many attempts. Timing and error rates were not recorded across a controlled set of trials, so the rows marked **[MEASURE]** in Table 7 and Table 9 are left for that work.

3.1 Unit and Integration Testing

Unit testing means checking small pieces of code one at a time. We wrote 18 of these tests for the server, where the decisions are made. They cover three things: that passwords follow the rules, that the warning checks work on a quiet record as well as a busy one, and that the date maths is right.

Integration testing means checking that two parts agree with each other. The website is checked against the server automatically. That check passed with no errors across 4,871 files. This catches the case where the server starts sending something the website does not expect.

The reader firmware was compiled for the target board before flashing. Both programs built with no errors, which confirmed that the modules link correctly and that the libraries match the board.

3.2 System Testing

System testing means using the whole thing the way a person would. We ran the server and the website together against a database that already held records. This matters. An empty database returns nothing, so a fault that only appears with real data would stay hidden.

We logged in first, on purpose. It worked and returned the right account. This proved the system was alive, so any later failure would mean something real and not just a dead server.

Then we opened each page the website uses. The record list, the borrow list, the active session check, and the live feed all worked. The record list held 39 saved events of eight different kinds. All 20 pages of the website loaded.

3.2.1 Testing the Assembled Readers

With both readers built and flashed, the complete sequence was exercised on the hardware itself. A registered finger at the door released the door lock and opened a session. The same finger at the cabinet released the cabinet within that session. The cabinet displayed its one-time code, and the website accepted a borrow record only when that code was entered.

The refusal paths were exercised as well, since a system that only ever grants access has not been tested. A registered finger presented at the cabinet without a door session first was refused, and the refusal was written to the record as a tailgating attempt. A finger that had expired out of its session was likewise refused. In both cases the cabinet stayed shut and the attempt appeared in the trail with a name, a reader, and a time.

3.3 Test Results

Table 7 records the outcome of every activity described above, together with the evidence behind it. Rows marked **[MEASURE]** are behaviours already shown to work whose performance has not yet been quantified.

Table 7*What we tested and what happened*

What we tested	Result	How we know
<i>Software</i>		
Server unit tests	Passed	18 tests, none failed
Website and server fit	Passed	0 errors, 4,871 files
Door reader build	Passed	Compiled, no errors
Cabinet reader build	Passed	Compiled, no errors
Logging in	Passed	Correct account returned
Record list	Passed	39 events, 8 kinds
Borrow list	Passed	Records returned
Session check	Passed	Worked
Live feed	Passed	Stayed connected
Website pages	Passed	20 of 20 loaded
<i>Assembled hardware</i>		
Door scan opens session	Passed	Lock released, session created
Cabinet opens for same person	Passed	Released inside the session
One-time code shown	Passed	Displayed on opening
Borrow record needs code	Passed	Refused without it
Tailgating refused	Passed	No session, cabinet stayed shut
Expired session refused	Passed	Cabinet stayed shut
Refusals recorded	Passed	Written to the trail
<i>Not yet quantified</i>		
Scan time	[MEASURE]	Works; not timed over trials
Unlock time	[MEASURE]	Works; not timed over trials
False acceptance rate	[MEASURE]	Needs a controlled trial set
False rejection rate	[MEASURE]	Needs a controlled trial set
Text messages	Stand-in only	No real SIM card

We found one fault, and we report it instead of quietly fixing it, because it shows why testing with real data matters. The server sent all 39 records, but the website showed none of them. The reason: the server had learned two new kinds of record that the website did not know about. The website checks the whole list at once, so one unknown item threw away all 39. The spreadsheet page, which checks each row on its own, showed everything. That difference is what told us where the fault was. The saved records themselves were never harmed.

3.4 System Requirements

Table 8 states the requirements the system was designed against, separated into what it must do and how it must behave. These are the statements the tests above were written to check.

Table 8*What the system must do*

No.	Requirement
<i>What it must do</i>	
F1	The door opens only for a finger signed up on the door sensor.
F2	A door scan starts a session for that one person, which ends by itself.
F3	The cabinet opens only for the person holding a running session.
F4	Every decision is saved with the person, the reader, and the time.
F5	The cabinet shows a one-time code, and no borrow record is accepted without it.
F6	The website shows which readers are working.
<i>How it must behave</i>	
N1	Readers never decide on their own.
N2	Fingerprints never leave the sensor.
N3	Nobody can edit or delete a saved reader record, not even an administrator.
N4	Each reader has its own password, which can be changed on its own.
N5	The program must fit in the chip with room to spare.
N6	The reader must never freeze while the lock is open or a message is showing.

Every requirement in the table was exercised during testing. F1 to F6 were demonstrated on the assembled readers, as recorded in Table 7. N1, N2 and N4 follow from how the system is built and were confirmed by inspection: a reader holds no decision logic, no template leaves the sensor, and each reader carries its own credentials. N3 was confirmed by attempting to edit a reader record through the interface, which offers no route to do so. N5 is measured below. N6 was observed on the hardware, where a second finger could be presented while a message was still on screen.

3.5 Performance Analysis

Performance is examined here in two ways. The first is what the finished program demands of the chip, which the build reports exactly. The second is how the design sits against systems that have already been measured and published.

3.5.1 Use of Chip Resources

Our program fills about 81 percent of its 1.2 MB space. That meets requirement N5, but only just. Adding features later will be limited by this, not by speed or memory.

RAM is not a problem, and the reason is a design choice. Fingerprints stay in the sensor, so the biggest piece of data never reaches our chip. That turns the usual tightest limit on this kind of project into a non-issue. It also answers the small-memory warning raised by Rahman et al. (2026).

Speed is not a problem either. Espressif Systems (2025) rate the chip at 1079.96 CoreMark, a standard speed score. Our reader only waits on a slow sensor, builds a short message, and keeps Wi-Fi alive. Two processors mean the Wi-Fi keeps running while the sensor is read.

3.5.2 Timing

Table 9 separates the intervals fixed by the design, which are known exactly, from those that depend on the sensor and the network and would have to be timed over repeated trials.

Table 9*Timings we set, and timings still to be measured*

Interval	Value	Status
Session length	120 s	Set in the server
Checkout code lifetime	900 s	Set in the server
Enrollment code lifetime	300 s	Set in the server
Door lock stays open	3 s	Set in the reader
Cabinet lock stays open	4 s	Set in the reader
Message on screen	2.5 s	Set in the reader
“Still here” signal	30 s	Set in the reader
Scan time	[MEASURE]	Observed to work; not timed
Unlock time	[MEASURE]	Observed to work; not timed

3.5.3 Comparison With Published Systems

The figures below belong to other people’s systems. They are the benchmarks this system should be compared against once its own numbers are recorded.

Alsayaydeh et al. (2025) measured a scan at about one second and an unlock at five seconds on a setup much like ours. They admitted the wrong person 1.32 percent of the time and turned away the right person 26.32 percent of the time. The second figure is the warning. A system that

rejects one honest person in four will end up propped open, and a propped door defeats everything behind it.

Mohammed et al. (2025) report a mean authentication time of 235 milliseconds and a 68 percent drop in fraudulent access. Their timing is not a fair comparison, since their pipeline runs on server-class machines rather than a microcontroller. The 68 percent is the relevant figure: it is the scale of improvement that checking twice is expected to deliver, and the effect the door-to-cabinet sequence is built to reproduce.

Two properties of this system have no counterpart in either study. Refusals are recorded as carefully as approvals, so tailgating attempts become a countable quantity rather than an assumption, and the refusal tests above confirm those records are written. Reader events are also append-only by construction, which is the accountability property Koisser and Sadeghi (2023) argue a record must have before it can be used as evidence.

CHAPTER 4: CONCLUSION

4.1 Problems Encountered

Six problems shaped the final design. The first three appeared in the software, where they were found and corrected. The next two were electrical, and were caught while the readers were being assembled. The last belongs to the room itself.

4.1.1 Slot Numbers Meant Different People

Each sensor keeps its own list of fingerprints. Slot 7 on the door sensor and slot 7 on the cabinet sensor are two different people. Our first design treated slot 7 as one person everywhere. That would have opened the cabinet for the wrong person, and the record would have looked perfectly normal.

4.1.2 A Reader Cannot Be Trusted to Decide

At first each reader decided by itself and then told the server. That is easy to build and wrong. A reader hangs on a wall. Anyone can open it and copy or fake its messages. Anything that decides who gets in has to be decided by the server.

4.1.3 One Unknown Record Emptied the Whole Page

Described in Chapter 3. The server learned two new kinds of record. The website did not know them, so it threw away all 39 records instead of just the two. The page looked empty, and no error was shown.

4.1.4 Settings Were Cut Without Warning

The server reads its settings from a text file. That file quietly cuts anything after a space. A name with a space in it gets shortened, and nothing tells you.

4.1.5 Electrical Problems That Only Appear in Hardware

Three problems surfaced while the readers were being wired up. The lock pulls a lot of current when it opens and will restart the chip if the two share one supply. The text unit does the same when it sends. And one of its wires cannot take the chip's full signal strength. Each of these shows up as a random restart, which is hard to trace back to its cause.

4.1.6 The Door Reader May Not Reach the Wi-Fi

The door is the farthest point from the router. A reader that cannot reach the server cannot ask permission, and by design it will not decide alone. So it opens nothing.

4.2 Troubleshooting

This section describes how the problems above were traced. In each case the useful part was not the fix but the observation that made the fault visible in the first place.

4.2.1 Finding the Empty Page Fault

The server replied correctly with a full page of data, so the server and the network were ruled out at once. The clue was that the spreadsheet page showed all 39 records while the record page showed none. Both read the same data, so the difference had to be in how each one read it. Comparing the kinds of record in the data against the kinds the website accepted found the two unknown ones.

The wider lesson is what made it visible at all: we tested with real data. With an empty database both pages show nothing, and the fault would have looked like normal behaviour.

4.2.2 Always Start With Something That Works

Every round of testing began with a request we knew should succeed. Without that, a test where everything fails only proves the server is off. It says nothing about the parts being tested.

4.2.3 Checking Code Before Flashing

Before the boards were flashed, the only available checks were the ones the tools can make: that the code compiles, that the modules link, and that the image fits the chip with measurable room left. Those checks caught real mistakes early, but they cannot show behaviour. Everything about how the system actually behaves came afterwards, from the assembled readers.

4.3 Solutions and Conclusions

This section sets out what was changed in response to those problems, what the study achieved, and what it was not able to show.

4.3.1 What We Fixed

Finger records now carry the name of the reader they came from, so a slot number can never be matched against the wrong sensor. The cost is that people sign up twice, once at each reader. We accepted that cost rather than hide it.

All decisions moved to the server. Readers log in with their own ID and password, not a person's, so a stolen reader gives away only itself. Reader records were made permanent, with no way to edit or delete them for anyone, including administrators. A wrong record is corrected by adding a right one.

The electrical problems were solved in the wiring. The lock and the text unit each got their own power supply, sharing only a ground wire with

the chip, and the text unit's weak wire got a voltage reducer. Where the door reader cannot reach the Wi-Fi, the cabinet reader can share its connection.

The empty page fault is still open. The fix is to teach the website all the kinds of record the server sends, and to check each row on its own so one unknown item cannot throw away a whole page.

4.3.2 Conclusion

This project set out to prove who took materials from a school stockroom, and to record it so nobody can change it afterwards.

The design goals were met. We documented how the chip's processors, memory, pins, and Wi-Fi work, and showed that each one shaped a real decision. The clearest example is memory. By leaving fingerprints in the sensor, the largest piece of data never reaches the chip. What is usually the tightest limit on this kind of project stopped being a limit at all.

The software goals were met and tested. The two-step check, the session, the permanent record, and the code-locked borrow list all work, and Chapter 3 shows the results. Safety comes from the shape of the design, not from one clever part. Decisions sit in one place, readers can only speak as themselves, and refusals are saved as carefully as approvals.

The main goal was demonstrated. Both readers were built, flashed, and run through the full sequence. The cabinet refused a finger that held no door session, and the attempt was written to the record. What remains is measurement rather than function. Until scan time, unlock time, and error rates are recorded over repeated trials, this system cannot be set beside the figures reported by Alsayaydeh et al. (2025), and the 68 percent reduction that Mohammed et al. (2025) measured cannot be confirmed in this setting.

Two limits remain even now that the hardware is built. The sensor does the matching, so we inherit its accuracy and have no way to spot a fake finger; the seaweed-jelly fakes shown by Rad et al. (2024) would get through. And the borrow list is still typed by a person. We can prove the cabinet opened and who opened it, but not that the typed list is complete. Closing that gap would mean sensing the items themselves, which is a different project.

4.3.3 Recommendations

The following work would close the gaps this study leaves open. The first item matters most, because it is the difference between a system shown to work and a system whose performance is known.

1. Record scan time, unlock time, and error rates over a controlled set of trials, so the system can be compared against published figures.
2. Fix the empty page fault before the system is used for real.
3. Test with a real SIM card and a real phone network.
4. Consider a sensor with fake-finger detection, given the weakness shown by Rad et al. (2024).
5. Watch the refusal count once the system runs. It shows how often tailgating is actually tried.

References

- Alrawili, R., AlQahtani, A. A. S., & Khan, M. K. (2024). Comprehensive survey: Biometric user authentication application, evaluation, and discussion. *Computers and Electrical Engineering*, 119, 109485. <https://doi.org/10.1016/j.compeleceng.2024.109485>
- Alsayaydeh, J. A. J., Yusof, M. F., Mamchenko, S., Hamzah, R. A., & Herawan, S. G. (2025). Design and evaluation of a biometric IoT-based smart lock system with real-time monitoring and alert mechanisms. *International Journal of Advanced Computer Science and Applications*, 16(7). <https://doi.org/10.14569/IJACSA.2025.0160737>
- Cepeda, J. A. U., & Saludes, A. J. C. (2025). Online ICT equipment inventory and borrowing system with decision support. *European Journal of Innovative Studies and Sustainability*, 1(5), 62–70. [https://doi.org/10.59324/ejiss.2025.1\(5\).07](https://doi.org/10.59324/ejiss.2025.1(5).07)
- Espressif Systems. (2025). *ESP32 series datasheet* (Version 5.3). https://www.espressif.com/sites/default/files/documentation/esp32_datasheet_en.pdf
- Koisser, D., & Sadeghi, A.-R. (2023). *Accountability of things: Large-scale tamper-evident logging for smart devices*. arXiv: 2308.05557 [cs.CR]. <https://arxiv.org/abs/2308.05557>
- Linuwih, H. W., & Handayati, Y. (2025). Quantitative analysis of inventory record inaccuracy (IRI): A case study on warehouse stock discrepancies. *International Journal of Current Science Research and Review*, 8(1). <https://doi.org/10.47191/IJCSRR/V8-I1-26>

- Lkhluf, R., Orcero, D. S., & Gonz'alez Cañete, F. J. (2026). Blockchain-assisted authentication, authorization, and audit for MQTT-based smart-city IoT. *Future Internet*, 18(9), 463. <https://doi.org/10.3390/fi18090463>
- Mohammed, A., Salama, A., Shebka, N., & Ismail, A. (2025). Enhancing network access control using multi-modal biometric authentication framework. *Engineering, Technology & Applied Science Research*, 15(1), 20144–20150. <https://doi.org/10.48084/etasr.9554>
- OWASP Foundation. (2026). *Session management cheat sheet* [Continuously maintained; last revised August 2026]. Retrieved September 11, 2026, from https://cheatsheetseries.owasp.org/cheatsheets/Session_Management_Cheat_Sheet.html
- Rad, P., Dorai, G., & Jozani, M. (2024). *From seaweed to security: The emergence of alginate in compromising IoT fingerprint sensors*. arXiv: 2404.02150 [cs.CR]. <https://arxiv.org/abs/2404.02150>
- Rahman, M. H., Shihab, M., Rahman, M. A., & Naderuzzaman, M. (2026). ESP32 microcontroller: A review of architecture, communication protocols, applications and research challenges. *Open Access Journal on Engineering Applications*, 1(2), 19–33. <https://doi.org/10.64886/oajea.0102.003>
- Yagiz, M. A., Horasan, F., & Yurttakal, A. H. (2026). *Lightweight tamper-evident log integrity verification for IoT edge environments: A Merkle tree pipeline with adaptive chunking*. arXiv: 2605.00065 [cs.CR]. <https://arxiv.org/abs/2605.00065>